Loading and Displaying JSON Data

JavaScript Fetch API and jQuery $.getJSON()

THEORY AND CASE STUDY | PRACTICAL 10

Aim

To load and display JSON data using the JavaScript fetch() API and jQuery $.getJSON(), and render the results dynamically in an HTML table.

Software/Tools Required

- HTML5 for the page structure and semantic data table.
- CSS3 for table styling, layout, and responsive presentation.
- JavaScript ES6+ and the Fetch API for asynchronous JSON requests.
- jQuery for the $.getJSON() shorthand request method.
- VS Code or any text editor, and Live Server for local development.

Theory

1. JSON and HTTP GET requests

JSON (JavaScript Object Notation) is a lightweight text format for exchanging structured data. A client sends an HTTP GET request to a JSON resource, receives a response, and parses the response body into JavaScript values before displaying it.

2. Loading data with fetch()

The Fetch API returns a Promise for a network response. Calling response.json() parses its body asynchronously. A reliable request flow checks response.ok, handles network or parsing errors, and reports a useful status to the user.

3. Loading data with jQuery $.getJSON()

jQuery $.getJSON(url) is a shorthand for an asynchronous GET request expecting JSON. Its done callback receives the parsed data, while fail can display a request error. Both APIs can supply the same renderer.

4. Dynamic table rendering

JavaScript can create table rows and cells from an array of objects, then append them to a table body without reloading the page. A clear empty state and safe text insertion keep the output understandable and prevent data from being interpreted as markup.

Practical 10 | Case Study and Implementation

Case Study Overview

This case study builds a JSON data viewer that requests a collection of records from a JSON endpoint and presents the returned fields as table rows. Two loading controls demonstrate equivalent asynchronous workflows: native fetch() and jQuery $.getJSON(). The table is populated only after valid JSON data is received.

Key Features Implemented

- Load structured JSON from a remote endpoint or local JSON file.
- Demonstrate both fetch() and jQuery $.getJSON() GET requests.
- Render each record dynamically as a table row and cells.
- Show loading, empty-result, and request-error feedback.
- Use responsive table styling for different screen widths.

Core Implementation Details

HTML Structure (index.html)

- Add controls for loading data and a semantic table with a header and tbody target.
- Include a status area for loading, empty, and error messages.

CSS Styling (style.css)

- Style the table, headings, row spacing, and status messages.
- Use responsive overflow or layout rules for narrow displays.

JavaScript Functionality (script.js)

- Request JSON with fetch(), check response.ok, and parse using response.json().
- Request the same data with $.getJSON() and handle success/failure callbacks.
- Pass returned records to a shared renderer that creates cells and inserts text safely.
- Clear stale rows before rendering and display an empty state when no records are returned.

Conclusion

This JSON viewer demonstrates asynchronous data retrieval using both native JavaScript and jQuery. Parsing the response and rendering each record into a table connects HTTP requests with practical DOM manipulation, while status and error handling make the page more usable.